

Пояснительная записка к проекту

по курсу: Структуры и алгоритмы обработки данных

по теме: Сравнительный анализ алгоритмов триангуляции для задачи поиска пути на навигационных мешах

организация: Филиал МГУ имени М.В. Ломоносова в городе Дубне

направление: Методы и технологии обработки данных в гетерогенных вычислительных средах

выполнил: студент 5-го курса Шипунов Григорий Александрович

дата: 4 июня 2026 г.

Введение

Задача поиска пути является одной из центральных в вычислительной геометрии, робототехнике и разработке компьютерных игр. Её суть состоит в нахождении геометрически оптимального маршрута для агента, перемещающегося в двумерном пространстве с препятствиями. Классические подходы, основанные на сеточных разбиениях и графах видимости, либо страдают от высокой вычислительной стоимости, либо порождают пути, далёкие от оптимальных. Навигационный меш (*Navigation Mesh*, NavMesh) — современная альтернатива: пространство свободного перемещения триангулируется, каждый треугольник становится узлом графа, а поиск ведётся уже по этому графу [1].

Триангуляция — разбиение множества точек на непересекающиеся треугольники, объединение которых покрывает выпуклую оболочку исходного множества. Качество триангуляции влияет на несколько характеристик навмеша одновременно: суммарный вес (сумма длин всех рёбер), форма треугольников (узкие «иглы» ухудшают численную стабильность), а также топология графа, по которому будет выполняться поиск пути. Наиболее изученная триангуляция — **триангуляция Делоне**: для любого треугольника его описанная окружность не содержит других точек множества. Это условие максимизирует минимальный угол и хорошо изучено теоретически [2]. Вместе с тем задача минимизации суммарного веса рёбер — так называемая *минимально-весовая триангуляция* (MWT) — является NP-трудной [3]. Для её практического решения разработаны эвристики: жадная триангуляция и квази-жадный алгоритм Левкопулоса – Кржнарника [4].

Цель настоящего проекта — реализовать три алгоритма триангуляции (Боуера – Уотсона, жадный, квази-жадный), построить на их основе навигационные меши, применить алгоритм A* для поиска коридора треугольников и алгоритм «воронки» (*Funnel Algorithm*) для нахождения кратчайшего пути, а затем сравнить все три подхода по ключевым метрикам на нескольких тестовых сценах.

Описание алгоритмов и структур данных

Двусвязный список рёбер (DCEL). Все триангуляции хранятся в структуре DCEL (*Doubly Connected Edge List*). Каждое ненаправленное ребро (u, v) представлено двумя полурёбрами: h (от u к v) и h^* (от v к u). Каждое полуребро хранит указатели на исходную вершину, двойника, следующее/предыдущее полуребро и принадлежащую грань. Такая структура обеспечивает обход граней, звёзд вершин и перестройку топологии за $O(1)$ на операцию. Индексы элементов стабильны при добавлении — удаление помечается флагом `dead` без реального освобождения памяти, что исключает инвалидацию ссылок во время перестройки.

Алгоритм Боуера – Уотсона (Delaunay). Инкрементальная вставка точек по одной. Для каждой вставляемой точки p : выполняется поиск «плохих» треугольников, чьи описанные окружности содержат p ; извлекается граница звёздообразной полости; все плохие грани уничтожаются; полость перетриангулируется соединением p с каждым ребром границы. На завершающем шаге супертреугольник и все инцидентные ему грани удаляются. Результат является триангуляцией Делоне [5].

Жадная триангуляция (Greedy MWT). Генерируются все $\binom{n}{2}$ кандидатных рёбер; они сортируются по длине. Ребро принимается, если оно не пересекает ни одно уже принятое ребро (собственное пересечение). Принятые рёбра образуют триангуляцию [6]. Затем из множества рёбер извлекаются треугольные грани и строится DCEL.

Квази-жадная триангуляция (Quasi-Greedy MWT). Алгоритм Левкопулоса – Кржнарника [4]:

1. Строится триангуляция Делоне, рёбра которой служат кандидатами для MWT.
2. Для каждого ребра (p, q) проверяется условие пустой луны ($\beta=1$): открытый диск с диаметром pq не содержит других точек. Эквивалентно: $\forall r \neq p, q: (r - p) \cdot (r - q) \geq 0$. Такие рёбра гарантированно входят в MWT.
3. Незаполненные «карманы» триангулируются жадно по диагоналям.
4. При включённом режиме выполняются проходы перекидывания рёбер: ребро (u, v) между треугольниками (u, v, w) и (v, u, x) заменяется на (w, x) , если $|wx| < |uv|$.

Навигационный меш (NavMesh). Каждая живая ограниченная грань DCEL становится узлом графа; смежные грани соединяются дугами. Вес дуги — евклидово расстояние между центроидами. Для каждой дуги хранится «портал» — пара вершин общего ребра, используемая алгоритмом воронки.

Алгоритм A*. Поиск кратчайшего по сумме весов дуг коридора треугольников от стартового узла до целевого. Эвристика — евклидово расстояние между центроидами; она допустима, поскольку веса дуг сами являются этими расстояниями [7].

Алгоритм воронки (Funnel Algorithm). Преобразует коридор треугольников в кратчайший евклидов путь за $O(k)$, где k — число треугольников в коридоре. Поддерживаются две цепочки (левая и правая), описывающие воронку видимости из текущей вершины. При обработке каждого портала цепочки обновляются; при пересечении воронки вершина «апекса» смещается и фиксируются промежуточные точки пути [8].

Теоретическое обоснование выбранных решений

Выбор DCEL в качестве центральной структуры данных обусловлен тем, что все операции триангуляции (перекидывание рёбер, разбиение граней, обход окрестности) сводятся к локальным перелинковкам указателей без перестройки всей структуры. Это принципиально важно для алгоритма Боуера – Уотсона, который при каждой вставке затрагивает лишь $O(\deg)$ граней.

Алгоритм Делоне выбран в качестве эталонного, поскольку его результат единственен (при отсутствии кокруговых точек), хорошо изучен теоретически и широко применяется в промышленных навмешах. Жадная триангуляция включена как простой базовый вариант MWT-эвристики. Квази-жадный алгоритм — единственная известная полиномиальная эвристика с доказанной константной аппроксимацией MWT [4]; он использует Делоне как источник кандидатов, что практически устраняет избыточные рёбра.

Для поиска пути A* с евклидовой эвристикой является оптимальным выбором на планарных графах: эвристика допустима и монотонна, что гарантирует нахождение оптимального по стоимости дуг пути без повторного раскрытия узлов [7]. Алгоритм воронки обеспечивает теоретически оптимальный евклидов путь через коридор треугольников за линейное время.

Анализ вычислительной сложности

DCEL. Операции вставки вершины, перекидывания ребра, поиска грани по точке: $O(1)$, $O(1)$, $O(n)$ соответственно. Пространственная сложность: $O(n)$ вершин, $O(n)$ рёбер, $O(n)$ граней — всего $O(n)$.

Алгоритм Боуера – Уотсона. Средняя временная сложность при случайных точках — $O(n \log n)$ (каждая вставка обрабатывает $O(1)$ плохих граней в среднем); в наихудшем случае (регулярные решётки, точки на одной окружности) — $O(n^2)$. Пространственная сложность — $O(n)$.

Жадная триангуляция. Генерация $O(n^2)$ кандидатов и их сортировка — $O(n^2 \log n)$. Проверка пересечений для каждого кандидата с уже принятыми рёбрами — $O(n^2 \cdot k)$, где $k = O(n)$, итого $O(n^3)$ в худшем случае. Доминирует сортировка: $O(n^2 \log n)$. Память: $O(n^2)$ (список кандидатов).

Квази-жадный алгоритм. Делоне-фаза: $O(n \log n)$ в среднем. Тест пустой луны — $O(n)$ на ребро, $O(n^2)$ всего. Триангуляция карманов — $O(n \log n)$ на практике. Каждый проход перекидываний — $O(n)$. Итоговая сложность: $O(n^2)$ (определяется тестом луны). Память: $O(n)$ (граф Делоне плюс DCEL вывода).

NavMesh. Построение: $O(n)$ (один проход по граням). Память: $O(n)$ (по числу граней).

A*. Временная сложность: $O((V + E) \log V)$, где $V \approx 2n$, $E \approx 6n$ — итого $O(n \log n)$. Память: $O(n)$.

Алгоритм воронки. $O(k)$, где k — длина коридора. Память: $O(k)$.

Результаты работы алгоритмов

Тестирование проводилось на четырёх наборах данных различного масштаба: **points.txt** (8 точек — тривиальный случай), **tavern.txt** (22 точки — небольшая сцена с препятствиями), **labyrinth.txt** (80 точек — лабиринт со множеством барьеров) и **labyrinth_big.txt** (798 точек — крупная нагрузочная сцена). Все измерения выполнены на одном запуске; время построения может варьироваться в зависимости от загруженности системы, поэтому при анализе важны порядок величин, а не абсолютные значения.

Таблица 1: points.txt — 8 точек, маршрут (0.001, 0) → (1.999, 0)

Алгоритм	Треугольников	Вес	Построение, мс	Длина пути	Поворотов
Делоне (Боуер–Уотсон)	8	17.94	0.4	2.65	1
Жадный MWT	8	17.94	0.1	2.65	1
Квази-жадный MWT (Л.–К.)	8	17.94	0.5	2.65	1

Таблица 2: tavern.txt — 22 точки, маршрут (0.5, 0.5) → (9.5, 9.5)

Алгоритм	Треугольников	Вес	Построение, мс	Длина пути	Поворотов
Делоне (Боуер–Уотсон)	38	191.59	0.5	15.59	2
Жадный MWT	38	186.40	0.8	12.97	1
Квази-жадный MWT (Л.–К.)	38	186.40	1.5	12.97	1

Таблица 3: labyrinth.txt — 80 точек, маршрут (0.5, 0.5) → (19.5, 19.5)

Алгоритм	Треугольников	Вес	Построение, мс	Длина пути	Поворотов
Делоне (Боуер–Уотсон)	154	792.04	3.5	35.11	13
Жадный MWT	154	749.16	40.4	35.20	14
Квази-жадный MWT (Л.–К.)	154	756.32	34.9	33.78	7

Таблица 4: labyrinth_big.txt — 798 точек, маршрут (0.5, 0.5) → (49.5, 49.5)

Алгоритм	Треугольников	Вес	Построение, мс	Длина пути	Поворотов
Делоне (Боуер–Уотсон)	1590	5677.42	107	85.05	29
Жадный MWT	1590	5475.05	20578	81.55	29
Квази-жадный MWT (Л.–К.)	1590	5492.22	17854	87.46	37

Анализ результатов. Для всех наборов данных каждый алгоритм дал одинаковое количество треугольников после триангуляции, что подтверждает корректность их работы, так как данное значение подчиняется формуле Эйлера (с учётом внешней оболочки графа):

$$N = 2V - S - 2, \quad (1)$$

где N — число треугольников, V — число вершин, S — число вершин, входящих в оболочку.

На малом наборе (8 точек) алгоритмы дают идентичные пути, и различаются только по времени построения, которое для каждого из алгоритмов пренебрежимо мало.

На средних сценах (22–80 точек) жадный и квази-жадный алгоритмы дают более лёгкую триангуляцию, чем Делоне. На лабиринте (80 точек) квази-жадный алгоритм обеспечивает путь с наименьшим числом поворотов (7 против 13–14) при сопоставимой длине: это практически важный результат, поскольку частые смены направления увеличивают нагрузку на управляющую логику агента.

На нагрузочной сцене (798 точек) разрыв во времени построения становится ощутимым: Делоне справляется за 107 мс, тогда как жадный и квази-жадный требуют соответственно $\approx 20,6$ и $\approx 17,9$ секунды. Это непосредственное следствие квадратичной сложности: жадный алгоритм проверяет $O(n^2)$ пар рёбер, а квази-жадный выполняет $O(n)$ тестов луны на каждое из $O(n)$ рёбер Делоне. Оба алгоритма неприемлемы для больших сцен без применения пространственного индекса.

Заключение

В рамках проекта разработана и реализована система, включающая три алгоритма триангуляции (Боуера – Уотсона, жадный, квази-жадный), двусвязный список рёбер (DCEL) как центральную структуру хранения, навигационный меш на основе DCEL, алгоритм A^* для поиска коридора треугольников и алгоритм воронки для построения кратчайшего евклидова пути. Реализовано консольное приложение для сравнительного бенчмарка и SVG-визуализатор результатов.

Ключевые выводы по результатам экспериментов следующие. Алгоритм Делоне является наилучшим выбором при строгих требованиях ко времени построения: его практическая скорость на 2–3 порядка выше, чем у MWT-эвристик при $n \approx 800$. Жадный и квази-жадный алгоритмы дают более лёгкие триангуляции и, как правило, более плавные пути, однако их временная сложность $O(n^2)$ делает их применение оправданным лишь для сцен малого и среднего размера (до ≈ 200 точек). Квази-жадный алгоритм превосходит жадный как по весу, так и по числу поворотов в итоговом пути, при сравнимом времени работы.

В качестве направлений дальнейшего развития можно выделить: внедрение пространственного индекса (kd-дерево или R-дерево) для ускорения теста пустой луны до $O(n \log n)$; поддержку динамического обновления навмеша при изменении сцены; расширение на трёхмерные триангуляции Делоне.

Список литературы

1. Mononen M. Recast & Detour Navigation Mesh Toolkit. [GitHub](#), 2009.
2. de Berg M., Cheong O., van Kreveld M., Overmars M. *Computational Geometry: Algorithms and Applications*. 3rd ed. Springer, 2008. DOI: [10.1007/978-3-540-77974-2](#).
3. Mulzer W., Rote G. Minimum-weight triangulation is NP-hard. *Journal of the ACM*, 2008, vol. 55, no. 2, pp. 1–29. DOI: [10.1145/1346330.1346336](#).
4. Levcopoulos C., Krznaric D. Quasi-greedy triangulations approximating the minimum weight triangulation. *Journal of Algorithms*, 1998, vol. 27, no. 2, pp. 303–338. DOI: [10.1006/jagm.1997.0938](#).
5. Bowyer A. Computing Dirichlet tessellations. *The Computer Journal*, 1981, vol. 24, no. 2, pp. 162–166. DOI: [10.1093/comjnl/24.2.162](#).
6. Gilbert P.D. New results on planar triangulations. Technical Report R-850. University of Illinois, 1979.
7. Hart P.E., Nilsson N.J., Raphael B. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 1968, vol. 4, no. 2, pp. 100–107. DOI: [10.1109/TSSC.1968.300136](#).
8. Lee D.T., Preparata F.P. Euclidean shortest paths in the presence of rectilinear barriers. *Networks*, 1984, vol. 14, no. 3, pp. 393–410. DOI: [10.1002/net.3230140304](#).